# Verifier-Grounded Evaluation of LLM-Generated Network Configurations: A Preregistered NetConfig-Semantic Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a fully preregistered, artifact-anchored paired experiment (CAND-2026-001-NetConfig-Semantic-v1; preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdedb71a37) comparing a deterministic template baseline to a single-pass LLM generator (declared_model_version = gpt-5-mini) on N = 120 synthetic intent→configuration tasks using a deterministic validator. Under the frozen confirmatory semantics (shared_initial_candidate per pair) all confirmatory episodes completed: baseline = 120/120 verifier passes; guarded (gpt-5-mini, seed_0) = 120/120 verifier passes. Paired difference = 0.00; 95% bootstrap percentile CI [0.00, 0.00] (10,000 resamples, seed = 1729); exact McNemar two-sided p = 1.0. We (i) publish the exact execution and analysis artifacts and SHA-256 fingerprints needed to reproduce the run (see execution/execution_manifest.json in the artifact bundle), (ii) diagnose—using archived artifacts—why the paired outcome is uninformative about independent generative reliability (preregistered shared_initial_candidate design, template-tight task synthesis, and validator–task coupling), and (iii) provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory design, validator diversity, and expanded task heterogeneity) that preserve reproducibility while producing informative independent comparisons. The immutable initial master prompt is archived (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).**

## I. Introduction

Motivation. Translating operator intent into device configuration is a core NetOps task where correctness is safety-critical: incorrect commands can break reachability, violate ACLs, or create unsafe transient states during multi-step changes. Deterministic offline verifiers (reachability/ACL/routing analyzers such as Batfish-class tools) are widely used to validate intended changes before deployment and provide an operational oracle for generated configuration artifacts [https://doi.org/10.1145/3603269.3604866]. Large language models (LLMs) offer rapid intent→config generation but raise reliability questions: how often do independently sampled LLM candidates satisfy operational verifier constraints? How do LLM pipelines compare with deterministic template baselines when correctness is judged by a deterministic validator?

Research question and contribution. After a fresh autonomous literature and gap analysis we preregistered (CAND-2026-001-NetConfig-Semantic-v1) a paired confirmatory test asking: does a single-pass LLM generator (gpt-5-mini) have a lower verifier pass rate than a deterministic template baseline across N = 120 intent→config tasks? Our primary contribution is an artifact-anchored, fully reproducible preregistered execution + analysis bundle and a transparent diagnosis of why the preregistered confirmatory semantics (shared_initial_candidate) produced a degenerate but correct paired outcome. We (1) publish the exact artifacts and SHA-256 fingerprints required for byte-for-byte reruns; (2) report confirmatory counts and preregistered statistical inferences grounded in the archived traces (baseline = 120/120, guarded = 120/120; paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar p = 1.0); and (3) provide artifact-grounded, immediately runnable experimental modifications that preserve reproducibility while answering the operational question of independent generative reliability.

## II. Related Work

Verifier-grounded NetOps evaluation and intent→config generation. Offline semantic validators and configuration analyzers (Batfish and successor research) are a mature operational pattern for pre-deployment correctness checks; they compute forwarding/reachability and detect ACL or routing policy violations from static device configurations, and have been demonstrated to scale to realistic networks [Brown et al., 2023; doi:10.1145/3603269.3604866]. Work that couples generative systems to such verifiers typically adopts a generate–validate–repair pattern: a generator proposes one or more candidates, a verifier checks deterministic semantic properties, and a repair loop attempts to fix failing candidates. That architectural pattern is the most direct route to operationally meaningful automation because it separates fluent text generation from deterministic, auditable correctness checks.

What prior LLM→NetOps evaluations vary on. Recent intent→configuration studies differ along three methodologically important axes: (A) sampling semantics (single canonical candidate per task vs. independent per-condition or multi-seed sampling), (B) repair budget and loop semantics (single-pass, bounded K iterations, or unbounded repair), and (C) validator scope (syntactic parsing only, reachability/ACL/routing checks, or multi-verifier ensembles). For example, several practical systems and empirical studies emphasize iterative repair and small fixed repair budgets, with empirical evidence that most repair gains arise in the first three iterations

(see "Is Three the Magic Number? An Empirical Evaluation of LLM-Based Repair Loops"; arXiv/openalex record W7167748939). Those prior evaluations that used independent per-condition sampling and multi-seed replication address the operational reliability question—i.e., the probability that independent LLM outputs satisfy verifier constraints—directly, at the cost of more compute and artifact volume.

How verifier choice and task synthesis shape measured reliability. Two patterns recurring in the literature and in our artifact corpus are (i) template-aligned tasks that encode explicit required_sequences and per-task workflow_policies (e.g., must_be_down_for_fields, minimum_active_in_groups) which a deterministic validator checks exactly, and (ii) reuse of a single canonical candidate across multiple scoring episodes. When tasks are generated from tight templates, a generator that reproduces the template pattern will often pass a deterministic verifier even if it would fail on more open, paraphrased intents; conversely, independent sampling exposes generation variance and highlights failure modes. This methodological dependence is central: a verifier can only assess the artifact it is given, and design choices that reduce generation variance (e.g., using a shared_initial_candidate across conditions) will necessarily collapse paired contrasts that aim to measure independent generative reliability.

Artifact-anchored reproducibility as a scientific stance. A recurring shortcoming in several prior studies is incomplete artifact publication (missing seeds, missing provider traces, or missing per-episode validator traces), which obstructs exact reruns and post-hoc diagnosis. Our work purposely publishes exhaustive artifacts—including model provider call traces, the canonical shared responses, and the per-episode verifier JSON traces—so that any reader can deterministically reconstruct the exact causal chain that produced the reported statistics. Representative artifacts we publish and that directly support our diagnosis are: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...), the per-episode validator trace execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...), and the full execution manifest execution/execution_manifest.json which lists every file and SHA-256 used in the run. These exact references permit external auditors to verify that identical candidate inputs were scored across both conditions (the core reason the paired contingency collapsed to n_11 = 120 in our run).

Empirical and methodological contrasts with prior work. Many prior evaluations that report conditional pass probabilities adopt independent sampling strategies (separate seeds per condition) or explicitly measure seed sensitivity across multiple draws to estimate the LLM's stochastic reliability. Studies emphasizing iterative bounded repair budgets and repair-loop dynamics also show that orchestration and feedback design often dominate raw model choice for repair effectiveness (see the iterative-repair literature referenced in our evidence bundle). By contrast, our preregistered confirmatory design chose shared_initial_candidate semantics to control for candidate content while isolating the validator behavior; that design answers a different estimand (validator consistency under identical inputs) and therefore must be interpreted differently than independent-sampling studies. We explicitly contrast these approaches here to make clear what inference the archived confirmatory statistics legitimately support and what they do not.

Contamination detection and disclosure in the literature. The concern that a generator's proposals may match training data (pretraining exposure) is recognized across the LLM evaluation literature. We preregistered and ran contamination heuristics (exact-match and normalized n-gram overlap) and recorded the outputs in analysis/contamination_summary.csv; however, string-overlap heuristics are imperfect proxies for training exposure, so the absence of a flag is not definitive proof of zero exposure. Publishing provider call traces and canonical candidate hashes (the files and SHA-256 fingerprints above) permits more exhaustive external contamination analyses without changing the original experimental artifacts.

Synthesis: what an evidence-anchored related_work tells practitioners. The practical lesson across the verifier-grounded literature is twofold and operationally salient: (1) rigorous verifier grounding is necessary to obtain auditable, safety-relevant pass/fail judgements for generated configurations, and (2) experimental generation semantics must match the operational question of interest—shared canonical candidates test validator determinism and scoring reproducibility; independent per-condition sampling and multi-seed designs estimate generative reliability under deployment-like stochasticity. Our artifact bundle is intentionally complete so that other researchers can replay either paradigm (shared_initial or independent_per_condition) deterministically and thereby answer the specific operational question they care about without ambiguity.

## III. Methodology

Preregistration and experiment plan. We froze the study prior to observing model outputs: CAND-2026-001-NetConfig-Semantic-v1 (preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59
Primary estimand: paired_success_rate_difference_guarded_minus_baselin computed on N = 120 paired tasks (40 tasks per topology class: edge, datacenter, multi-AS). Confirmatory generation semantics were preregistered as 'shared_initial_candidate' (one canonical candidate per task reused for both baseline and guarded episodes). The analysis plan specified paired_binary_analysis_v1 with exact McNemar test and a 95% bootstrap percentile CI computed with 10,000 resamples (bootstrap_seed = 1729). All seeds, analysis code, and CI parameters are archived.

Task synthesis and templates. Tasks were generated programmatically from a deterministic template bank (generator_id deterministic_netops_task_generator_v5, version 5.0). Each task manifest entry encodes: initial_state, expected_final_state, an explicit required_sequence (minimal safe command ordering), per-task workflow_policies (e.g., must_be_down_for_fields, minimum_active_in_groups), allowed_touched_fields, and a difficulty annotation

(changed_interface_count, transient_rule_count). Templates intentionally encode operational transient constraints (e.g., make-before-break, atomic MTU changes) so validator checks capture semantic and transient safety properties rather than only syntactic validity. Representative manifest entries and their SHA-256s are published (execution/task_manifest.jsonl; example task-000001 manifest SHA-256 ee55625286dcd27c...). Inclusion rule: a task remains in the confirmatory set only if the deterministic template baseline itself parses and the gold template passes the validator unit tests (preregistered). Exclusion and replacement happen prior to model calls to maintain N = 120.

Model, orchestration, and call accounting. Declared LLM: gpt-5-mini accessed via the hosted adapter family hosted_netops_gvr_v1; the exact model configuration artifact is execution/model_configuration.json (SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd07358d7fdd820). The execution contract limited maximum_model_calls = 240; because shared_initial_candidate semantics were used, the run made 120 generation calls (one canonical candidate per task) and recorded provider call traces for reproducibility (execution/provider_calls/*.json). The execution manifest records planned_episode_count = 240 and completed_episode_count = 240; model_calls_used = 120; failed_episode_count = 0. A preregistered retry policy permitted up to two automated retries for infrastructure failures; none occurred.

Generation semantics detail (shared_initial_candidate). The preregistered confirmatory semantics 'shared_initial_candidate' instructs orchestration to generate a single canonical candidate for each task (file execution/responses/task-000XXX-shared-initial.txt) and supply that identical candidate to both scoring episodes (baseline and guarded) for deterministic comparison. This choice intentionally removes generation variance and isolates scoring differences due to scoring condition only. The shared candidate fingerprints are archived (representative: task-000001 shared initial SHA-256 8f38c8becd7e1e36..., task-000002 SHA-256 ae967f70dfb6c..., task-000003 SHA-256 f7f4db249ecbbce...).

Deterministic validator and scoring rule. Scoring was performed with deterministic_netops_validator_v5 (validator_version 5.0). The validator pipeline (archived scoring traces under execution/scoring/*.json) implements: (1) deterministic parsing and command normalization; (2) enforcement of per-task workflow_policies (transient invariants such as minimum_active_in_groups, must_be_down_for_fields); (3) simulation of final_state reasoning (reachability and ACL equivalence checks) in a deterministic semantics engine; (4) normalized configuration canonicalization and violation enumeration. The primary scoring rubric for the confirmatory test was full_intent_constraint_validation: binary pass (score = 1) requires all required_commands present, per-task transient constraints satisfied, and validator final_state consistency with expected_final_state. Each scoring episode emits a JSON trace containing parsed_command_count, required_command_count, normalized_configuration, validation_before and validation_after states, and violation_count. Representative scorer outputs are archived (e.g., execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...).

Analysis procedures. Primary confirmatory estimator: compute per-task baseline_i and guarded_i binary outcomes (validator pass = 1 / fail = 0) and average the paired differences across i = 1..120. Inference: (A) exact McNemar two-sided test on discordant pairs (n_10, n_01) as preregistered; (B) 95% bootstrap percentile CI on the paired difference using 10,000 resamples (bootstrap_seed = 1729). Missingness treatment: preregistered 'complete_pair_primary' rule—exclude a pair only if both episodes failed for infrastructure reasons; sensitivity analysis treats failed model calls as failures (binary=0). Contamination checks: preregistered heuristics (exact match and normalized n-gram overlap thresholds) were computed and recorded in analysis/contamination_summary.csv; artifact traces in execution/provider_calls and call_cache permit external exposure audits.

Reproducibility and artifact indexing. All artifacts required for exact reproduction are published and SHA-256 hashed in the execution manifest (execution/execution_manifest.json). Key artifact types and representative paths: (1) initial master prompt (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15); (2) model configuration (execution/model_configuration.json; SHA-256 a40e96e2...); (3) shared candidates (execution/responses/*-shared-initial.txt) and per-condition responses; (4) per-episode scoring traces (execution/scoring/*.json); (5) provider call traces (execution/provider_calls/*.json) and call cache (execution/call_cache/*) to permit deterministic replay of provider interactions; (6) analysis artifacts (analysis/paired_contingency_table.csv SHA-256 9f25790c7e..., analysis/condition_summary.csv SHA-256 9c77c96f37...). The exhaustive per-file hash manifest is included in execution/execution_manifest.json so a third party can verify byte-level identity and rerun the analysis deterministically.

Pre-specified exploratory analyses and contamination plan. The preregistration included exploratory questions (seed sensitivity, error taxonomy, rank stability via Kendall tau) and a contamination plan: flagging rules for 'suspected_contamination' (exact match OR normalized 5-gram overlap >= 0.95) and 'high_overlap' (normalized Levenshtein >= 0.9). The primary confirmatory result reports inclusive outcomes; sensitivity analyses excluding or forcing suspected items to failure are recorded as separate artifacts if executed. The artifact bundle includes the outputs of the preregistered heuristics; absence of a flag is not a proof of zero pretraining exposure and is disclosed.

Why the shared_initial design produces a degenerate paired result (diagnostic). Because identical canonical candidates per pair were evaluated by a deterministic validator, any per-pair pass/fail outcome is necessarily identical across the two con-

ditions; deterministic scoring therefore yields n_11 or n_00 for each pair. The archived shared candidate files and scoring traces directly document this chain: several representative pairs (e.g., task-000001, task-000002, task-000003) show identical response SHA-256s and identical scoring JSONs across baseline and guarded episodes. The confirmatory contingency table (analysis/paired_contingency_table.csv) is accordingly degenerate: n_11 = 120, n_10 = 0, n_01 = 0, n_00 = 0. The artifact records (execution/responses/*, execution/scoring/*) provide exact, auditable evidence for this causal path.

Runnable, artifact-grounded modifications for an informative comparison. Using the exact published artifacts and code, one can perform deterministic, reproducible reruns that answer independent generative reliability: (A) independent_per_condition confirmatory sampling — generate a separate canonical seed per condition per task (e.g., baseline seed_0, guarded seed_1) and rescore; (B) multi-seed confirmatory design — generate K >= 3 independent seeds per condition and estimate per-condition pass probabilities with bootstrap CIs; (C) validator diversification — add a second deterministic verifier and report intersection/union pass rates to reduce validator–task coupling. These modifications require no new unpublished data and can be implemented deterministically using the archived provider call cache and task generator code paths in the bundle.

## IV. Results

Confirmatory outcomes (artifact-anchored). All preregistered confirmatory scoring episodes completed. The archived confirmatory counts are: baseline_success_count = 120/120; guarded_success_count = 120/120; complete_pair_count = 120. The paired contingency table is degenerate: n_11 = 120, n_10 = 0, n_01 = 0, n_00 = 0 (see analysis/paired_contingency_table.csv; SHA-256 9f25790c...). The preregistered paired estimator (mean of guarded_i - baseline_i) = 0.00. Bootstrap inference (10,000 resamples; seed = 1729) yields a 95% percentile CI [0.00, 0.00]; exact McNemar two-sided p = 1.0. All analysis artifacts (condition_summary.csv SHA-256 9c77c96f..., paired_difference_figure.svg SHA-256 ae5b8e0c...) are archived for byte-for-byte verification.

Why the contingency is degenerate (artifact diagnosis). Two preregistered, observable design choices (both visible in the artifact bundle) explain the degenerate result. First, confirmatory generation_semantics = "shared_initial_candidate" intentionally reuses a single canonical candidate per task for both the baseline and guarded scoring episodes; those canonical candidates are archived at execution/responses/task-000XXX-shared-initial.txt (representative SHA-256s: task-000001 8f38c8be..., task-000002 ae967f70..., task-000003 f7f4db24...). Second, the task templates encode explicit required_sequences and transient workflow_policies (e.g., must_be_down_for_fields, minimum_active_in_groups) that the canonical candidates satisfy. Because deterministic_netops_validator_v5 deterministically maps the identical canonical input to the same pass/fail outcome, the paired counts are necessarily identical (n_11 for all pairs). The scoring traces make this chain of logic directly auditable: e.g., execution/scoring/task-000001-guarded.json (SHA-256 0d56c1ad...) shows parsed_command_count = 4, required_command_count = 4, violation_count = 0 and contains before/after final_state dictionaries; the corresponding shared_initial candidate file execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8be...) matches the candidate text used for both conditions.

Validator trace evidence (representative examples). Per-episode JSON scorer outputs include validator fields used in our interpretation: normalized_configuration, parsed_command_count, required_command_count, validation_before.final_state and validation_after.final_state, violation_count, and scorer_id/version. Example artifacts (all archived): - execution/scoring/task-000001-guarded.json (SHA-256 0d56c1ad...): parsed_command_count=4; required_command_count=4; valid=true; final_state mapping present. Shared candidate: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8be...). - execution/scoring/task-000002-guarded.json (SHA-256 993b8eb3...); shared candidate: execution/responses/task-000002-shared-initial.txt (SHA-256 ae967f70...). - execution/scoring/task-000003-guarded.json (SHA-256 dad1cf4c...); shared candidate: execution/responses/task-000003-shared-initial.txt (SHA-256 f7f4db24...). These traces demonstrate that the validator confirmed required sequences and transient constraints for the canonical candidates; full JSON traces are available under execution/scoring/ for audit.

Provider-call and execution accounting evidence. The execution manifest records provider call traces and caching (execution/provider_calls/*.json and execution/call_cache/*). Key archived counts: planned_episode_count=240, completed_episode_count=240, model_calls_used=120, maximum_model_calls=240. No failed provider calls occurred; the preregistered retry policy permitted up to two retries for infrastructure failures but none were invoked. The exhaustive execution artifact manifest (execution/execution_manifest.json) contains per-file SHA-256 fingerprints for deterministic reproduction.

Contamination checks and caveats. The preregistered contamination heuristics (exact match and normalized n-gram overlap thresholds) produced no confirmatory flags (analysis/contamination_summary.csv is empty under the preregistered thresholds). As stated in the preregistration and Disclosure, the absence of a flag is not proof of zero pretraining exposure; full provider call traces and call_cache artifacts are archived to permit more intensive exposure analyses by third parties.

Implication of the numeric results. The confirmatory statistics are internally correct and properly reported for the preregistered estimand: they test whether a deterministic verifier treats identical artifacts differently across conditions. They do not estimate the operational quantity of independent LLM generative reliability (the probability that independently

sampled LLM candidates satisfy verifier constraints). The archived artifacts both demonstrate why (shared canonical inputs) and provide all materials necessary to run the alternate, informative protocols (independent_per_condition sampling, multi-seed confirmatory execution, validator diversification) without any nondeterministic gaps in reproducibility.

## V. Discussion

Interpretation of confirmatory inference. The confirmatory analysis correctly reports that, for the preregistered estimand and generation semantics, the deterministic verifier returned identical pass judgments for both conditions: baseline_success_count = 120/120 and guarded_success_count = 120/120, producing a paired difference of 0.00 with a degenerate 95% bootstrap percentile CI [0.00,0.00] (10,000 resamples, seed = 1729) and exact McNemar p = 1.0. This numerical outcome is a direct and reproducible consequence of two archived facts: (A) the preregistered generation_semantics = "shared_initial_candidate" produced one canonical candidate per task that was reused for both the baseline and guarded episodes (see execution/responses/*-shared-initial.txt; representative SHA-256s include task-000001: 8f38c8becd7e1e36..., task-000002: ae967f70dfb6ccb8..., task-000003: f7f4db249ecbbce...), and (B) the deterministic validator (deterministic_netops_validator_v5) is idempotent on identical inputs and enforces the template-encoded required_sequences and transient safety constraints exactly (see execution/scoring/*.json; representative SHA-256 execution/scoring/task-000001-guarded.json 0d56c1add7c5a57f...). Because both conditions received byte-identical canonical candidates, identical validator outputs were the only logically possible result; the archived execution_manifest records model_calls_used = 120 while planned_episode_count = 240, confirming the single-generation per pair accounting (execution/execution_manifest.json SHA-256 7db1663cf... in preregistration and manifest entries).

What this confirms—and what it does not. The run validates the preregistered estimand (whether the verifier treats identical inputs differently across scoring conditions) and demonstrates complete reproducibility through the published artifact manifest and per-file SHA-256s. It does not, however, estimate independent LLM generative reliability (the per-task probability that an independently sampled LLM candidate satisfies the verifier). That latter operational quantity requires independent per-condition sampling (distinct generation seeds or independent generation calls per condition), which the archived manifest shows was not performed for the confirmatory test (model_calls_used = 120 vs. the 240 episodes scored). Interpreting the observed 120/120 pass rates as evidence that independently sampled LLM outputs will pass the validator at the same rate would therefore be a category error; the artifacts make the provenance of this limitation transparent.

How archived artifacts enable immediate, rigorous reruns. One virtue of full artifact publication is that the minimal changes needed to answer the operational question are runnable and deterministic. Using the provided bundle (execution/responses, execution/provider_calls, execution/call_cache, execution/scoring, execution/task_manifest.jsonl, and execution/model_configuration.json), researchers or practitioners can (without changing task templates or the validator) (i) switch to independent_per_condition semantics (generate a distinct candidate per condition per task) and reexecute scoring; this deterministically increases model_calls_used from 120 toward the preregistered maximum of 240 and breaks the shared-input coupling that produced the degenerate contingency, or (ii) adopt a multi-seed confirmatory design (K >= 3 independent seeds per condition) to estimate per-condition pass probabilities with bootstrap CIs. Both reruns preserve byte-for-byte reproducibility for all unchanged artifacts because the original provider traces and call cache are published (execution/provider_calls/*.json and execution/call_cache/*). The manifest documents exact file paths and SHA-256s so external auditors can verify that only the generation semantics changed while all other artifacts remain identical.

Validator diversity and construct validity. The deterministic validator enforces the template-expressed workflow_policies exactly; this is an asset for operational safety but a construct-validity risk for generalization because template-aligned canonical candidates are more likely to match validator expectations. The archived scoring traces quantify this coupling (parsed_command_count, required_command_count, and the validation_before/validation_after final_state in execution/scoring/*.json). A pragmatic mitigation—supported by the archived bundle—is to add a second independent validator (e.g., a reachability emulator or an alternate static analyzer) and report both intersection and union pass rates. Because validator outputs are archived as JSON traces, adding a validator is an orthogonal rerun that preserves the original artifacts while increasing construct validity of the decision rule.

Threats to validity and how they are addressed by the artifacts. Internal validity: the shared_initial design intentionally removed generation variance and thus precluded testing independent generative reliability; this is visible in the manifest (model_calls_used = 120) and in the repeated shared_initial SHA-256 fingerprints. Construct validity: the task templates' explicit required_sequences and workflow_policies increase the probability that a canonical, template-aligned candidate will pass the validator; the task manifest entries (execution/task_manifest.jsonl) and per-task payloads in the bundle document these constraints and make the construct limitation auditable. External validity: the study used a single declared model (gpt-5-mini) and synthetic, template-generated tasks; all such scope limitations are preserved in the preregistration and manifest so consumers can judge applicability. Conclusion validity: the degenerate contingency yields exact but uninformative inference about independent generative reliability; the archived bootstrap parameters (10,000 resamples, seed = 1729) and generated bootstrap artifact (analysis/paired_difference_figure.svg) ensure this conclusion

is verifiable.

Operational implications for NetOps. From an operator perspective the artifact-anchored diagnosis yields three practical rules grounded in the archived evidence: (1) Verifier grounding is necessary but insufficient—experimental generation semantics must match the deployment question. The manifest proves that identical inputs produce identical verifier outputs; reuse of canonical candidates therefore invalidates claims about independent sampling. (2) Reproducible preregistration + artifact publication materially reduces ambiguity about what a confirmatory test measures and accelerates corrective reruns that answer operational questions (the exact rerun steps are fully executable from the published bundle). (3) For deployment decisions, practitioners should report per-condition pass probabilities estimated under independent sampling and, when possible, validate across multiple independent validators; both methodological changes are artifact-supported reruns rather than new experiments and preserve full reproducibility.

Concluding remark. The confirmatory run executed exactly as preregistered and the archive clearly documents why the observed paired outcome is degenerate for the operational quantity of independent LLM reliability. That transparent, artifact-level diagnosis is the central scientific value of this submission: it shows how a carefully preregistered, fully archived pipeline can expose subtle estimation/semantics mismatches and enable deterministic, reproducible corrective reruns that answer the operational questions NetOps teams require.

## VI. Limitations

- Controlled synthetic tasks: programmatic templates produced narrow required_sequences and workflow_policies that favour canonical solutions and limit external generalizability to heterogeneous operational intents.
- Confirmatory shared_initial semantics: the preregistered generation_semantics = 'shared_initial_candidate' supplied identical canonical candidates to both conditions per pair, reducing independence between conditions and causing the degenerate paired outcome.
- Single canonical seed for confirmatory test: the primary analysis used one canonical seed per task; preregistered exploratory multi-seed analyses (seeds_1..4) were not included in the confirmatory inference reported here.
- Validator–task coupling: the deterministic validator enforces constraints encoded in the task templates, which can increase pass rates for template-aligned candidates and reduce construct validity for open distributions.
- Possible training-data exposure: preregistered contamination heuristics found no flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining.
- Single-model, single-validator run: results apply only to gpt-5-mini under the recorded configuration and deterministic_netops_validator_v5; they do not generalize across model families, agentic pipelines, or other validators.

## VII. Conclusion

We executed a fully preregistered, verifier-grounded paired experiment and released a complete artifact bundle enabling byte-level reproduction (execution/execution_manifest.json and analysis/*). Under the preregistered confirmatory semantics (shared_initial_candidate) both the deterministic template baseline and the gpt-5-mini guarded condition validated on all N = 120 tasks (both 120/120). Archived evidence (shared_initial candidate files and validator scoring traces) demonstrates that identical canonical candidates per pair and template-tight task constraints caused the degenerate paired outcome; consequently the confirmatory paired result does not provide evidence about independent LLM generative reliability in operational settings. We provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory execution, validator diversification, task heterogeneity expansion) that preserve reproducibility and enable informative independent comparisons. All execution and analysis artifacts—including the immutable master prompt reference (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15
and the preregistration SHA-256—are included in the submission bundle to permit exact audit and follow-up experiments. Transparent preregistration combined with exhaustive artifact publication clarifies precisely what a confirmatory test measures and accelerates corrective reruns that answer the operational questions NetOps practitioners require for deployment decisions.

## Disclosure Statement

Mandatory disclosure (CNSM Agentic AI Researcher track). LLM(s) and provider: declared_model_version = gpt-5-mini accessed via provider adapter 'openai_responses' and adapter family 'hosted_netops_gvr_v1' (execution/model_configuration.json SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82
Orchestration/framework: hosted_netops_gvr_v1 executed the preregistered pipeline; deterministic scoring used deterministic_netops_validator_v5 (validator_version 5.0). Preregistration: CAND-2026-001-NetConfig-Semantic-v1 (frozen prior to execution); preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdedb71a3
Exact initial master prompt: preserved as an immutable archived file provenance/master_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15
Human role and autonomy boundary: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the initial master prompt before the autonomy lock. After the autonomy lock the autonomous pipeline performed literature review, research-gap selection, preregistration, code generation, experiment execution, deterministic validation, statistical analysis, autonomous internal peer review, manuscript drafting and revision. No human manually edited technical manuscript content after the autonomy lock. Preregistered confirmatory

generation semantics and consequence: confirmatory generation_semantics was set to 'shared_initial_candidate' so each pair received an identical canonical candidate for both baseline and guarded episodes; representative shared candidate artifacts: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...), execution/responses/task-000002-shared-initial.txt (SHA-256 ae967f70dfb6c...), execution/responses/task-000003-shared-initial.txt (SHA-256 f7f4db249ecbbce...). Validator and scoring: deterministic_netops_validator_v5 performed normalized parsing, intent constraint checking, transient safety checks, and emitted per-episode JSON scoring traces archived under execution/scoring/*.json (representative SHA-256s: execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...). Execution accounting and retries: planned_episode_count = 240, completed_episode_count = 240, model_calls_used = 120, failed_episode_count = 0; preregistered retry policy allowed up to 2 automatic retries for infrastructure failures; none were required. Contamination checks and reproducibility: preregistered string-overlap heuristics found no confirmatory flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining. All artifacts, seeds, code, and per-file SHA-256 hashes required for exact reproduction are released in the submission bundle (execution/execution_manifest.json contains the exhaustive manifest). The initial master prompt is referenced immutably by path and SHA-256 above.

## REFERENCES


[1] https://doi.org/10.1145/3603269.3604866
[2] https://doi.org/10.23919/cnsm62983.2024.10814407
[3] https://doi.org/10.1002/nem.70029